



2026

GitHub Copilot além do autocomplete

Como desenvolver com chat, agentes e
contexto compartilhado

Gustavo Bigardi

CTO @ Clarus IT · Microsoft MVP





QUEM SOU

Gustavo Bigardi

CTO @ Clarus IT

Microsoft MVP · 7 vezes consecutivas

Também nesta MVP Conf, daqui a pouco: *Aspire na prática*

github.com/gustavobigardi · about.me/gbbigardi



O Copilot é tão bom quanto o contexto que você dá a ele.

E tão seguro quanto a revisão que você faz depois.

Quem aqui usa o Copilot só para completar a linha?

MVP

Copilot



O que vamos ver

01

Além do autocomplete

O que o Copilot virou em 2026 e quanto custa

02

Chat e contexto

Investigar um código que você não conhece, ao vivo

03

Refatorar e testar

Uma sessão do agente gerando testes, ao vivo

04

O repositório ensina

Instruções, agentes, skills e MCP, ao vivo

05

Agentes na nuvem

Uma issue vira PR; o Copilot revisa o Copilot

06

Limites e riscos

Onde ele erra, o que os estudos dizem, o que levar

Além do autocomplete

O Copilot nasceu completando linhas em 2021. Em 2026 ele conversa, edita vários arquivos, roda comandos, trabalha sozinho na nuvem a partir de uma issue e revisa PRs. O autocomplete continua lá, mas é a menor parte.

De completar linha a abrir pull request

2021

Autocomplete

Preview em junho de 2021, GA em 2022. Sugere a próxima linha ou função no editor. Ainda é a porta de entrada.

2023

Chat

Copilot Chat no editor e depois no GitHub.com: pergunta em linguagem natural com o código como contexto.

2024

Vários arquivos

Copilot Edits e Workspace: um pedido muda vários arquivos; você revisa o diff antes de aceitar.

2025

Agentes

Agent mode no editor, coding agent na nuvem (issue vira PR) e o Copilot CLI no terminal, em preview.

2026

App e créditos

CLI GA (25/02), coding agent vira cloud agent (01/04), AI Credits (01/06), GitHub Copilot app (17/06), plugins (12/08).

A mudança de 2025 é a que importa: o Copilot deixou de sugerir texto e passou a executar tarefas. E quem executa precisa de contexto e de revisão.

Vem mais em outubro: o GitHub Universe é em 28 e 29/10. O que você vê hoje é o Copilot de setembro de 2026.

Quatro jeitos de usar o mesmo Copilot

01**Completar**

Autocomplete e next edit suggestions no editor. Ilimitado nos planos pagos. Bom para o que você já sabe escrever.

02**Conversar**

Chat no editor, no GitHub.com, no celular. Explica, investiga, propõe. Você copia o que quer.

03**Delegar localmente**

Agente no app, no CLI ou no editor: edita vários arquivos, roda build e testes na sua máquina, pede permissão antes de cada comando.

04**Delegar na nuvem**

Copilot cloud agent: pega uma issue, trabalha num runner do GitHub e abre um PR. O code review comenta o PR.

O QUE MUDA DE UM PARA O OUTRO

Quem digita: você → ele

Quantos arquivos: um → todos

Quem roda comando: você → ele, com sua permissão

Onde: sua máquina → um runner do GitHub

Planos em setembro de 2026: tudo vira AI Credits

Plano	Preço/mês	Créditos inclusos	O que tem (e o que não tem)
Free	US\$ 0	cota limitada	2.000 completions/mês, chat e agentes limitados, só modelo auto. Sem cloud agent e sem code review
Pro	US\$ 10	1.500 (= US\$ 15)	Completions ilimitadas, cloud agent, code review, seleção de modelos
Pro+	US\$ 39	7.000 (= US\$ 70)	Modelos premium (Opus, Fable, GPT-6 Astra), agentes de terceiros
Max	US\$ 100	20.000 (= US\$ 200)	Mesmo catálogo do Pro+, prioridade em modelos novos, limites de gasto maiores
Business	US\$ 19/usuário	1.900/usuário	Pool na organização, políticas centrais, dados não treinam modelos, indenização de IP
Enterprise	US\$ 39/usuário	3.900/usuário	Exige GitHub Enterprise Cloud; acesso antecipado

1 crédito = US\$ 0,01

O custo é o preço do modelo vezes os tokens. Créditos não acumulam: zeram no dia 1 de cada mês.

Acabou o crédito?

Faz upgrade pagando a diferença, define um orçamento extra (pay-as-you-go) ou espera o mês virar. Sem orçamento, não há excedente.

Quanto custa uma pergunta? Menos do que você imagina

O que eu pedi na Lojinha (ensaio de 22/09, modelo auto, tier eficiência)	Tempo	Créditos
"O que este projeto faz e onde é calculado o frete? O código bate com a regra?"	16 s	0,34 (US\$ 0,003)
"Crie o projeto de testes xUnit... rode dotnet test e me mostre o resultado"	45 s	0,92 (US\$ 0,009)
Agente revisor: "Revise o diff do último commit" (rodou build e testes)	16 s	0,46 (US\$ 0,005)

O modelo é o que pesa. Com 50 mil tokens de entrada e 5 mil de saída: Claude Sonnet 5 ≈ 15 créditos; Claude Fable 5.1 ≈ 75 créditos. O modo auto tem três perfis: eficiência, equilíbrio e inteligência.

A regra para iniciante

- Comece em auto, tier equilíbrio
- Suba para inteligência só em tarefa grande ou quando o resultado veio raso
- Não tenha medo de gastar crédito. Tenha medo de aceitar sem ler

Chat e contexto

A primeira coisa que o Copilot faz melhor que o autocomplete é ler o que você não escreveu. Um projeto que você nunca viu, um bug que alguém relatou, uma regra escondida num if. Você pergunta; ele procura.

Demo 1: investigar um projeto que você não conhece

A Lojinha do MVP Conf: uma API em .NET com catálogo, estoque e pedidos

"O que este projeto faz e onde é calculado o frete?"

Ele lê o código, explica e aponta o que não bate com a regra

MVP

Code



Anatomia de um pedido que funciona

01

Objetivo

Um verbo e um resultado: "explique", "encontre", "crie testes para", "corrija". Um pedido por vez.

02

Contexto

O que ele não adivinha: o arquivo (@), a issue (#), a regra de negócio, o erro completo, o que já tentou.

03

Restrições

"Não edite nada", "sem pacotes novos", "não mexa em produção", "em .NET 10". É o que evita o estrago.

04

Formato

"Em 8 linhas", "uma tabela", "mostre o resultado do dotnet test". Você lê mais rápido e revisa melhor.

```
// fraco
Explica esse código

// funciona
Explique em 8 linhas o que este projeto faz e onde é calculado o frete.
O código do frete bate com a regra do comentário da classe? Não edite nada.
```

Por que funciona

Objetivo (explique, compare), contexto implícito (o repositório inteiro), restrição (não edite) e formato (8 linhas). Foi esse o pedido da demo.

De onde vem o contexto (e o que você controla)

Fonte	Quem coloca	Exemplo na Lojinha
O que você escreve no pedido	Você	A regra do frete, o erro completo, o que já tentou
Arquivos e issues que você anexa	Você (@arquivo, #issue)	@ServicoDePedidos.cs; #12 "Cancelar pedido"
O repositório inteiro	Ele (busca, leitura)	Achou o frete com um grep por "frete subtotal"
Instruções do repositório	O time, uma vez	.github/copilot-instructions.md: comandos, regras, convenções
Fontes externas via MCP	O time, uma vez	Microsoft Learn: "quando acaba o suporte do .NET 10?"
Memória do Copilot	Ele, com o tempo	Preferências e fatos do repositório que ele guarda entre sessões

Regra prática: o que é verdade só hoje vai no pedido; o que é verdade sempre vai no repositório. As três primeiras linhas são o bloco 2 desta palestra; as três últimas são o bloco 4.

Refatorar e testar

Agora o Copilot escreve. Uma sessão do agente cria arquivos, roda o build e os testes na sua máquina e mostra o diff. Você aprova cada comando e lê cada linha antes de aceitar.

Demo 2: gerar os testes e achar o bug

Uma sessão do agente cria o projeto xUnit e roda dotnet test

Sete passam, um falha: o pedido de exatamente R\$ 200,00 paga frete

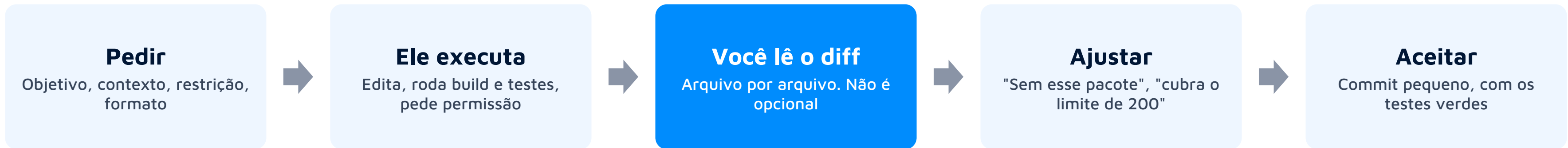
Você lê o diff antes de aceitar

MVP

Code



O ciclo que funciona com um agente



Modo da sessão	O que ele faz	Quando usar
Interactive	Propõe, edita e espera você antes de seguir; pede permissão para comandos	Sempre, enquanto você está aprendendo o que ele faz bem
Plan	Escreve um plano primeiro; você aprova; só então executa	Tarefa com mais de três arquivos ou que você não sabe explicar em duas frases
Autopilot	Escreve, roda testes e itera sozinho, sem esperar	Só com testes bons, sandbox ligado e repositório versionado

Pedidos que rendem em refatoração e testes

Testes

"Crie testes xUnit para ServicoDePedidos cobrindo o caso feliz e os limites: exatamente 200, estoque igual à quantidade, cupom em minúsculas"
Peça os limites por nome. É neles que o bug mora.

Refatorar

"Extraia o cálculo do frete e do desconto para métodos próprios sem mudar o comportamento. Rode os testes antes e depois"
Sem testes, refatoração é aposta. Peça os testes primeiro.

Investigar um erro

Cole a stack trace inteira: "Este erro aparece ao criar pedido com cupom. Encontre a causa e proponha a correção com teste"
Erro completo, não o resumo. Ele lê melhor que a gente.

Tarefas chatas

Mensagem de commit, descrição de PR, README, exemplo no .http, migração de versão, converter um JSON em record. É onde o ganho é maior e o risco é menor.

O que não pedir

"Melhore este código", "deixe mais rápido", "corrija tudo". Sem critério ele inventa um critério, e você não vai saber o que revisar.

Um pedido, um commit

Peça em pedaços que cabem num diff que você lê em dois minutos. Se o diff passou de 200 linhas, o pedido era grande demais.

O repositório ensina o Copilot

Tudo o que você repete em todo pedido ("em português", "rode os testes", "não use double para dinheiro") pode morar no repositório. Aí vale para você, para o time, para o app, para o CLI, para o editor e para o agente na nuvem.

Os arquivos que ensinam o Copilot

Arquivo	O que é	Vale onde
<code>.github/copilot-instructions.md</code>	Instruções do repositório: o que é, como compila e testa, regras e convenções	App, CLI, cloud agent, code review, chat no GitHub, editores
<code>.github/instructions/*.instructions.md</code>	Instruções que só valem para alguns caminhos (applyTo: "demo/**/*.cs")	Os mesmos, menos o chat no GitHub.com
<code>AGENTS.md</code>	O padrão aberto (também lido por outros agentes); o mais próximo do arquivo vence	App, CLI, cloud agent, code review, editores
<code>.github/agents/*.agent.md</code>	Agente custom: um prompt com nome, descrição e limites (ex.: revisor que não edita)	App, CLI, cloud agent, GitHub.com, editores
<code>.github/skills/*/SKILL.md</code>	Skill: passo a passo que ele carrega quando o pedido combina com a descrição	App, CLI, cloud agent, code review, editores
<code>.github/mcp.json</code>	Servidores MCP do repositório: fontes e ferramentas externas (Learn, GitHub, Playwright...)	App e CLI (o cloud agent tem a própria configuração)
Plugins e hooks	Pacotes com agentes, skills, hooks e MCP (github/awesome-copilot); hooks rodam scripts antes/depois de ferramentas	App, CLI, cloud agent

Tudo isso está no repositório da palestra. Copie a ideia, não o conteúdo: escreva o que é verdade sobre o seu código.

Demo 3: o repositório ensina o Copilot

Trago o .github: instruções, instrução só para testes, agente revisor, skill e MCP

"Corrija o frete seguindo a skill": ele testa antes, corrige, testa depois, responde no formato pedido

O agente revisor aprova; o MCP do Learn responde com fonte

MVP

Copilot



O que escrever nas instruções do repositório

Coloque

- O que o projeto é, em duas linhas
- Como compilar, testar e rodar (os comandos exatos)
- Onde ficam as coisas (pastas e classes principais)
- Regras de negócio que não estão óbvias no código
- Convenções: idioma, nomes, testes, o que nunca usar
- "Rode os testes antes de dizer que terminou"

Não coloque

- Segredos, tokens, URLs internas: vira contexto de todo pedido
- Ensaaios sobre arquitetura: ele lê tudo a cada pedido e paga tokens
- O que muda toda semana (número da sprint, quem está de férias)
- Instruções contraditórias entre arquivos: ele escolhe uma, e você não sabe qual
- Pedidos, só regras: "crie o endpoint X" não é instrução

Como começar

`copilot init` no CLI ou `/init` numa sessão: ele lê o repositório e escreve o primeiro rascunho

Revise como se fosse um README: apague o que não é verdade

Commit. A partir daí toda sessão de todo mundo nasce sabendo

MCP: como o Copilot deixa de inventar

Model Context Protocol

Um padrão aberto para o agente chamar ferramentas e fontes de fora: documentação, seu GitHub, um banco, um navegador

Servidor remoto é só uma URL (o Learn não pede nem login); servidor local é um processo (npx, docker)

O GitHub MCP já vem embutido no app e no CLI: issues, PRs, busca de código, sem configurar nada

Cada chamada aparece na sessão e pede permissão na primeira vez. Você vê o que ele consultou

```
// .github/mcp.json (commitado)
{ "mcpServers": {
  "microsoft-learn": {
    "type": "http",
    "url": "https://learn.microsoft.com/api/mcp"
  } } }
```

Cuidado

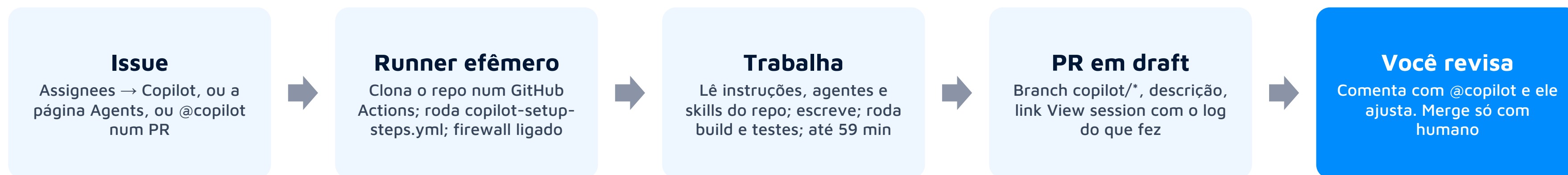
Servidor de projeto só carrega depois que você confia na pasta. Um MCP malicioso é um prompt injection com acesso a ferramentas: instale do registry oficial ou do seu time.

Na demo: "Consulte o Microsoft Learn: qual é a data de fim de suporte do .NET 10?" A resposta vem com a fonte, não com a memória do modelo.

Agentes na nuvem e revisão

Uma issue bem escrita entra; um pull request com testes sai. O Copilot cloud agent trabalha num runner efêmero do GitHub, com as mesmas instruções do repositório, e o Copilot code review comenta o que ele fez. Você continua sendo quem aprova.

Como o Copilot cloud agent trabalha



Uma boa issue para ele

Regras numeradas, o que devolver em cada erro, onde já existe exemplo, "teste primeiro". A issue da demo tem 8 linhas.

Custa

AI Credits da sessão + minutos do Actions (grátis em repositório público). Está em todos os planos pagos, inclusive o Pro. Não no Free.

Limites de segurança

Um repositório e uma branch por tarefa; não faz push em branch protegida; workflows do PR só rodam com aprovação sua; ambiente destruído no fim.

Demo 4: a issue virou pull request

"Cancelar pedido": endpoint, regras, devolver
estoque, teste primeiro

O PR do agente: descrição, commits, a sessão
com o log, os testes rodando

O Copilot code review comentou o PR do
Copilot

MVP

Copilot



Copilot code review: o Copilot revisa o Copilot (e você)

01**Como pedir**

Adicione "Copilot" como reviewer no PR, ou configure o repositório para revisar todo PR automaticamente ao abrir.

02**O que ele faz**

Comenta bugs, casos de borda e padrões; sugere o código; resolve o próprio comentário quando você atende; roda build e testes se puder.

03**Dois esforços**

Lite (rápido, US\$ 0,05 a 1) e Balanced (raciocina mais, US\$ 0,25 a 5). A partir de 28/09 o padrão vira Balanced.

04**Ele aprende com o repo**

Usa copilot-instructions.md, instruções por caminho, AGENTS.md e skills. As mesmas da Demo 3.

Ele pode até aprovar o PR (preview desde 01/09, desligado por padrão). Minha regra: revisor de IA é o primeiro par de olhos, nunca o último. Quem aprova entende o negócio.

Limites, riscos e revisão humana

Ele erra com confiança, não sabe o que não está no contexto, pode reproduzir código público, roda o comando que você aprovou sem ler e cobra por token. Nada disso impede o uso. Tudo isso exige revisão.

Onde ele erra (e como você percebe)

Alucina com confiança

Inventa método, pacote, versão. Percebe: build quebra, ou o pacote não existe no NuGet. Peça "rode o build" e MCP para docs.

Não sabe o que não viu

Regra que só está na cabeça do time, em outro repo, num e-mail. Percebe: o teste passa e o negócio está errado. Escreva a regra no repositório.

Faz o que você pediu, não o que você quis

"Corrija o teste" e ele muda o assert. Percebe: diff em arquivo que você não esperava. Restrinja o pedido.

Código igual ao público

Pode reproduzir trecho de repositório público mesmo com o filtro ligado. Percebe: licença estranha. Trate como código de terceiro.

Comando destrutivo

rm, git push --force, drop table: ele pede permissão, e você aprova no automático. Percebe: tarde. Leia antes de aprovar; nunca use "permitir tudo" fora de sandbox.

Você desaparece

Se nunca escreve o teste, esquece como. Percebe: não consegue revisar o que ele gerou. Alterne: uma feature com, uma sem.

O que os estudos dizem sobre produtividade

55% mais rápido

GitHub, 2022: 95 devs escrevendo um servidor HTTP em JavaScript. Com Copilot, 55% mais rápido e mais devs terminaram. Tarefa pequena, bem definida, sem código legado.

19% mais lento

METR, julho de 2025: 16 devs experientes em repositórios que conheciam há anos, com ferramentas de IA da época. Levaram 19% mais tempo, e achavam que tinham sido 20% mais rápidos.

As duas são verdade

Ganho grande em tarefa nova, pequena e bem descrita; ganho pequeno ou negativo em código que você domina e em pedido vago. E a sensação de velocidade engana.

Meça no seu time, não na palestra: tempo de PR, retrabalho, bugs em produção, e quanto do que ele gerou vocês aceitaram sem mudar. Os dashboards de Copilot da organização mostram uso; o resto é o seu processo.

Seus dados, seu código, sua licença

Pergunta	Free, Pro, Pro+, Max (individual)	Business e Enterprise
Meu código treina modelos?	Pode, desde 24/04/2026, salvo opt-out em Settings → Copilot	Não. Prompts são descartados depois da resposta
Quem vê meus prompts?	GitHub e o provedor do modelo; retenção zero na maioria dos modelos GA	Idem, com políticas centrais e content exclusions
Indenização de propriedade intelectual	Não	Sim, comprando direto do GitHub
Bloquear sugestões iguais a código público	Você liga na sua policy	O admin liga para todos
O agente na nuvem vaza código?	Firewall por padrão no runner; ambiente destruído no fim	Idem, mais políticas de repositório e de MCP

Para quem está começando: se o código é da empresa, o plano é da empresa. Se é seu, desligue o treino nas configurações hoje e nunca cole segredo no prompt.

Checklist de revisão humana

01**Antes de pedir**

Consigo dizer em uma frase o que quero e como vou saber que ficou certo? Se não, ainda não é hora de pedir.

02**Ao aprovar comando**

Eu li o comando? Ele mexe fora da pasta? Apaga, força push, chama a nuvem? Se sim, penso duas vezes.

03**Ao ler o diff**

Cada arquivo tocado era esperado? Os testes testam o negócio ou só passam? Tem pacote novo, double em dinheiro, segredo?

04**Antes do merge**

Rodei local? Entendo cada linha o suficiente para manter sozinho às 3 da manhã? A descrição do PR bate com o diff?

O QUE AJUDA A REVISAR

Pedidos pequenos

Testes que ele mesmo roda

Agente revisor que não edita

Code review no PR

/security-review antes do PR

**Dê contexto. Peça
pequeno. Leia tudo.
Revise sempre.**

O Copilot é tão bom quanto o contexto que
você dá a ele.

E tão seguro quanto a revisão que você faz
depois.

MVP

Copilot



Leve para casa

SEGUNDA-FEIRA

copilot init no seu repositório; revise o arquivo como um README e faça commit

PRÓXIMA FEATURE

Peça os testes antes do código e leia o diff inteiro antes de aceitar

TODO SPRINT

Olhe os créditos gastos e o que vocês aceitaram sem mudar; ajuste as instruções



**[github.com/gustavobigardi/
copilot-alem-do-autocomplete](https://github.com/gustavobigardi/copilot-alem-do-autocomplete)**
**A Lojinha, os arquivos de contexto, o roteiro
das demos e as referências conferidas**

Siga MVP Conf nas redes sociais



LINKEDIN



@MVPCONF

Obrigado!

Gustavo Bigardi

github.com/gustavobigardi · about.me/gbbigardi